Office Hour Gameplan: From Agent Idea → Deployable Plan

Today's outcome: a leadership-ready, 1-page proposal you can reuse



Today's Session

Office Hour Gameplan: From Agent Idea → Deployable Plan

Today's outcome

A leadership-ready, 1-page proposal you can reuse

Why this matters

Agents create value when they plug into real workflows + tools

Flow

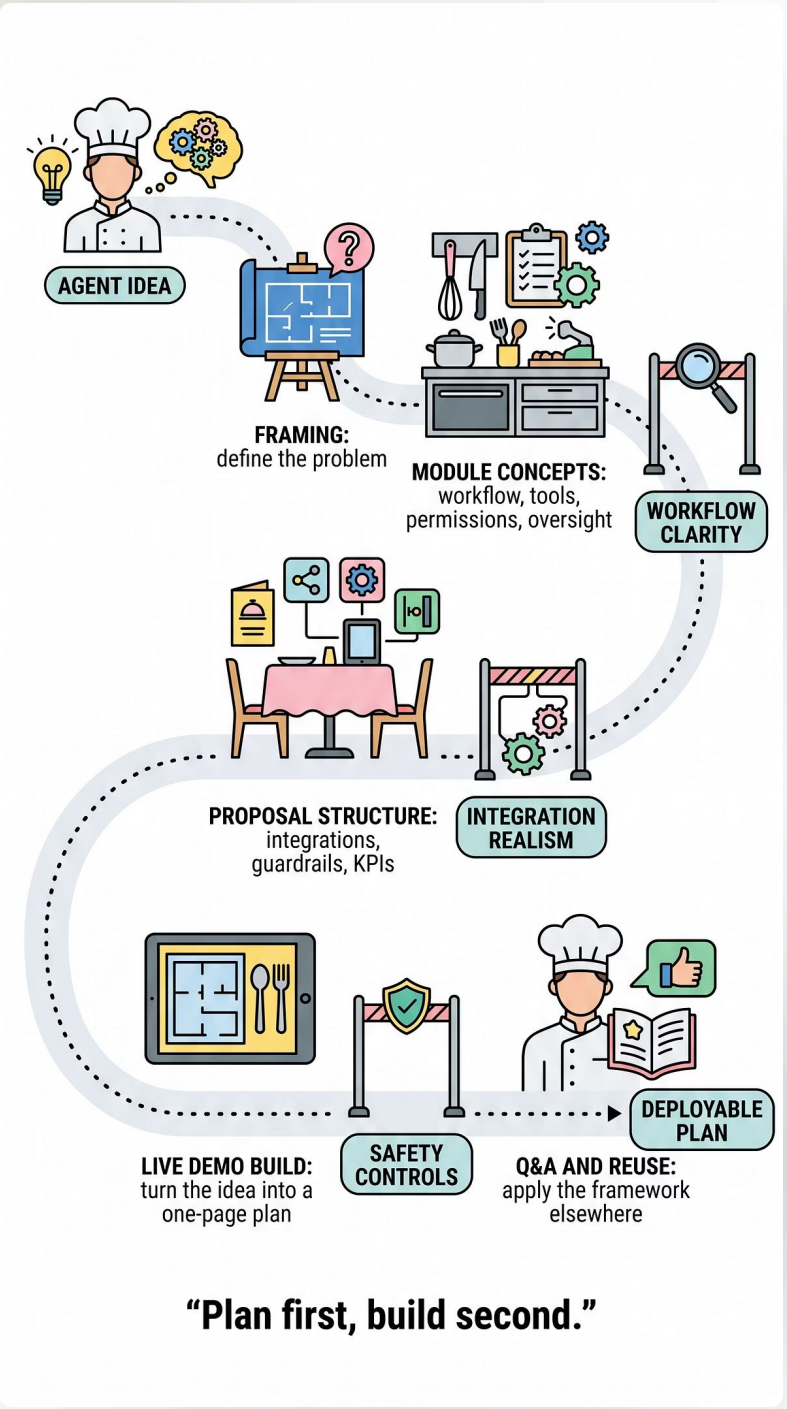
Framing → module concepts → proposal structure → live demo build → Q&A

You do not need to code today to learn the core adoption skill

What to watch for: workflow clarity, integration realism, and safety control points

We'll keep it company-agnostic so anyone can apply it

Metaphor (restaurant): we write the "opening plan" before we open the doors



The Big Shift: Chatbots Talk, Agents Operate

A chatbot answers questions

An agent helps work move forward

"Agentic" means

Can take steps, use tools, and coordinate actions

Real adoption requires

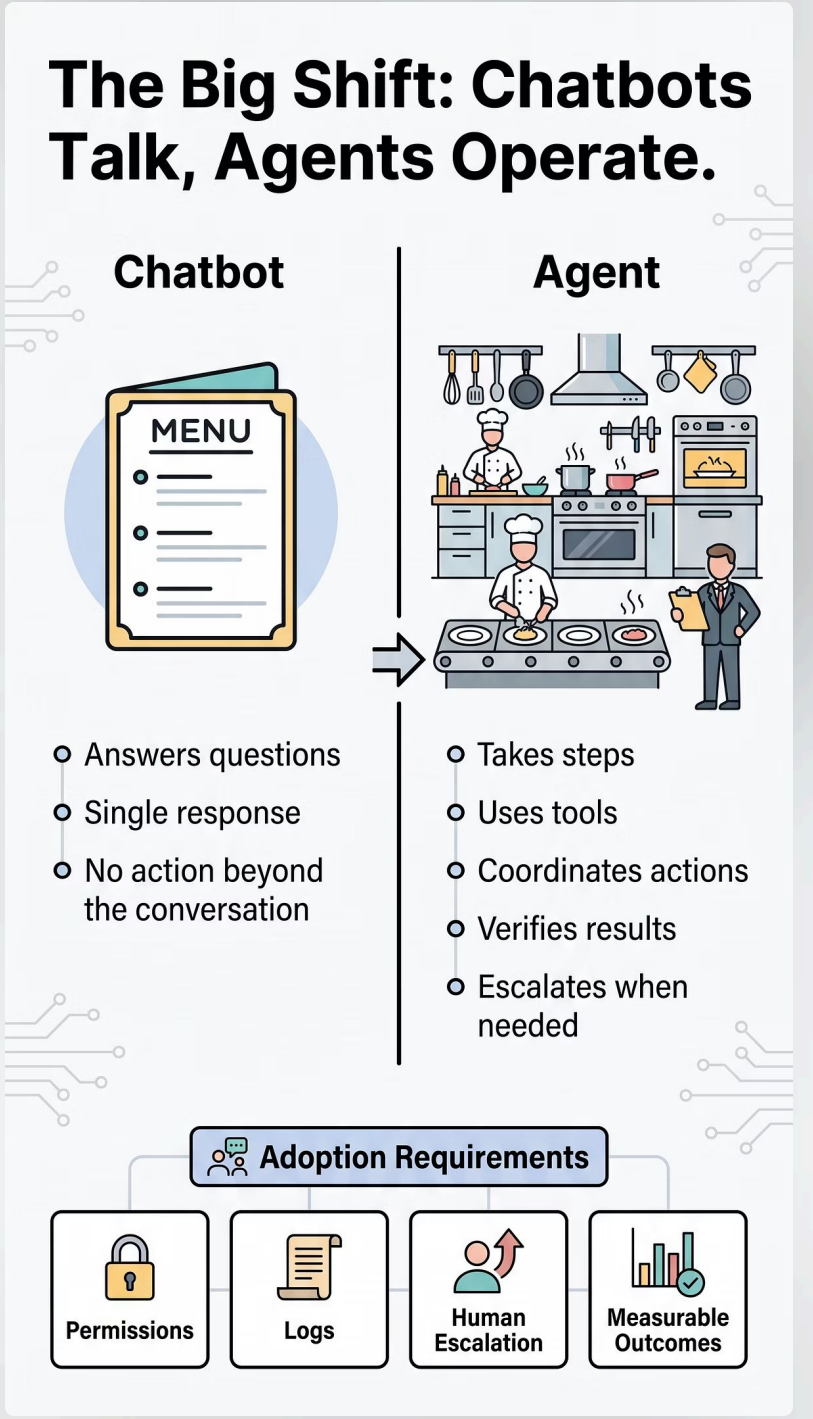
Permissions, logs, escalation, and measurable outcomes

Most failures aren't "bad AI" — they're unclear workflows + weak integrations

Today we practice the deployable path: plan first, build second

This mirrors how leaders actually approve AI work

Metaphor (restaurant): a menu is not a restaurant—operations are



Plain-English Definition: What Counts as an Agent

Agent = model + tools + rules + logs

So actions are explainable

It runs a loop

Trigger → decide → act → verify → escalate

Start in "suggestion mode"

Before earning "write access"

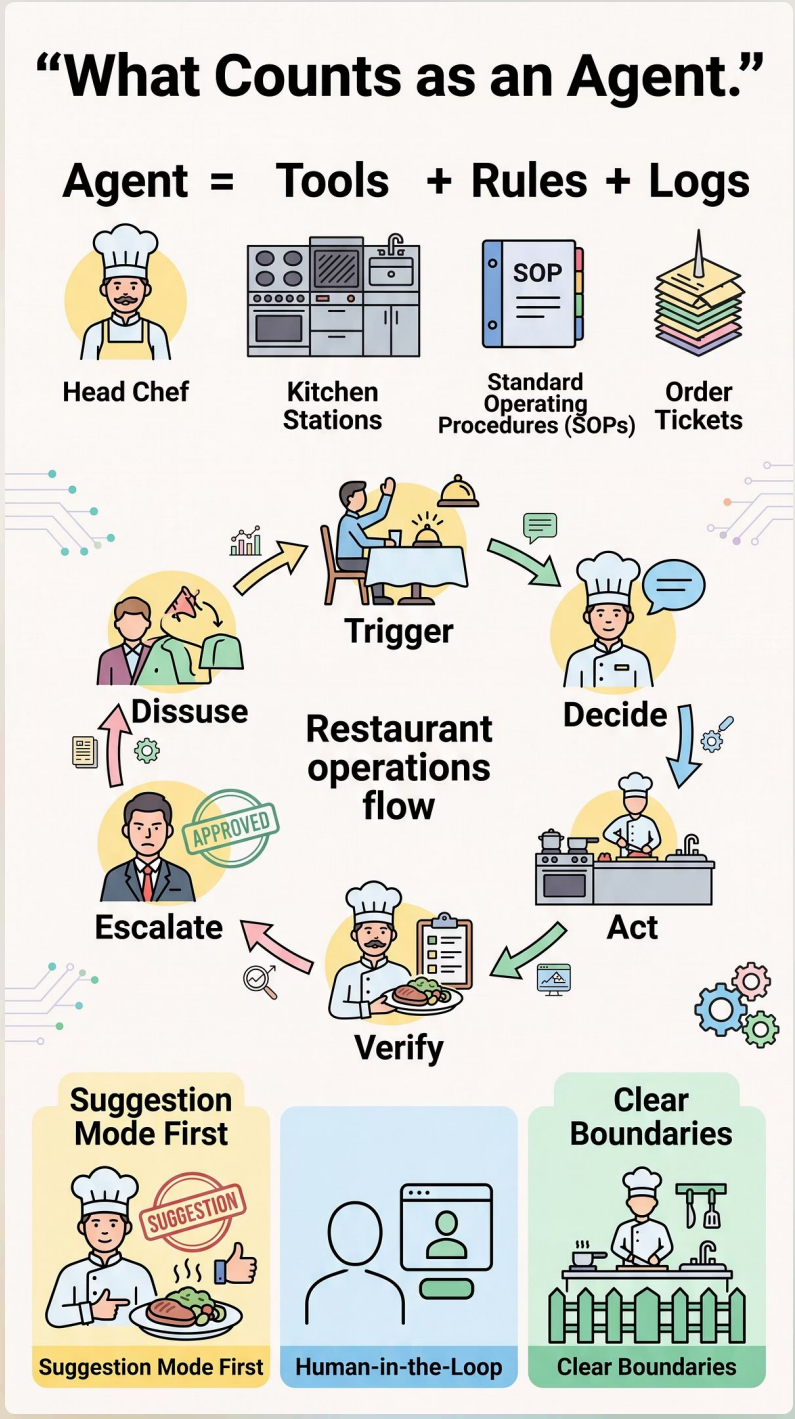
Human-in-the-loop is a feature

It's how you de-risk deployment

A good agent has boundaries: what it will do + what it refuses to do

If you can't describe failure modes, you can't ship responsibly

Metaphor (restaurant): staff + stations + SOP + manager escalation



Assignment Alignment: Two Options, One Core Skill

Option 1

Build a simple bot (implementation mechanics)

Option 2

Write an email-style proposal to leadership (adoption mechanics)

Today's office hour targets Option 2 because it generalizes to any org

Leaders approve

Workflows + integrations + guardrails + KPIs

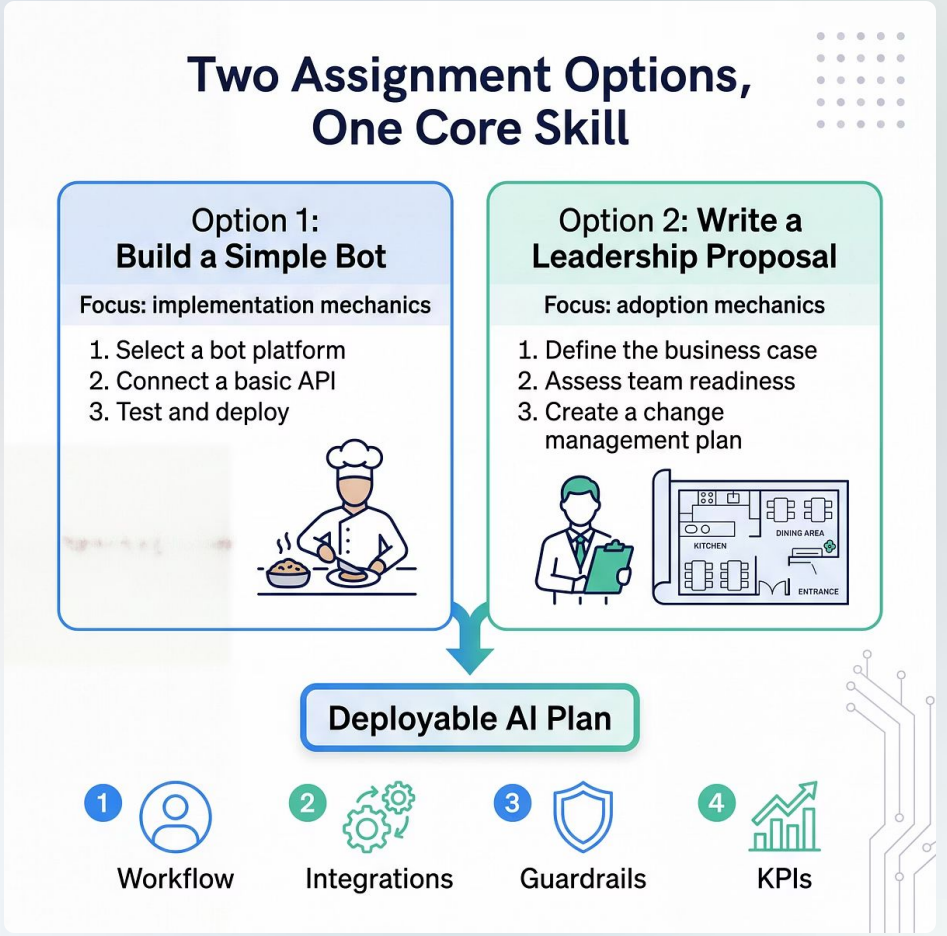
This is "business-ready"

Even for non-technical professionals

If you do Option 1

This proposal is still how you'd justify deploying it

Metaphor (restaurant): cooking one dish vs designing the whole service



The Demo Path: From Case Evidence to Pilot Decision

01

Define the Agent

Prompt 1: Set the assistant's job, workflow, and limits.

02

Map the Systems

Prompt 2: Identify the systems, information, and permissions needed.

03

Stress-Test the Incident

Prompt 3: Test the plan against conflicting policies and an unavailable billing lookup.

04

Finalize the Recommendation and Handoff

Prompt 4: Set owners, three targets, human approvals, and stop rules.

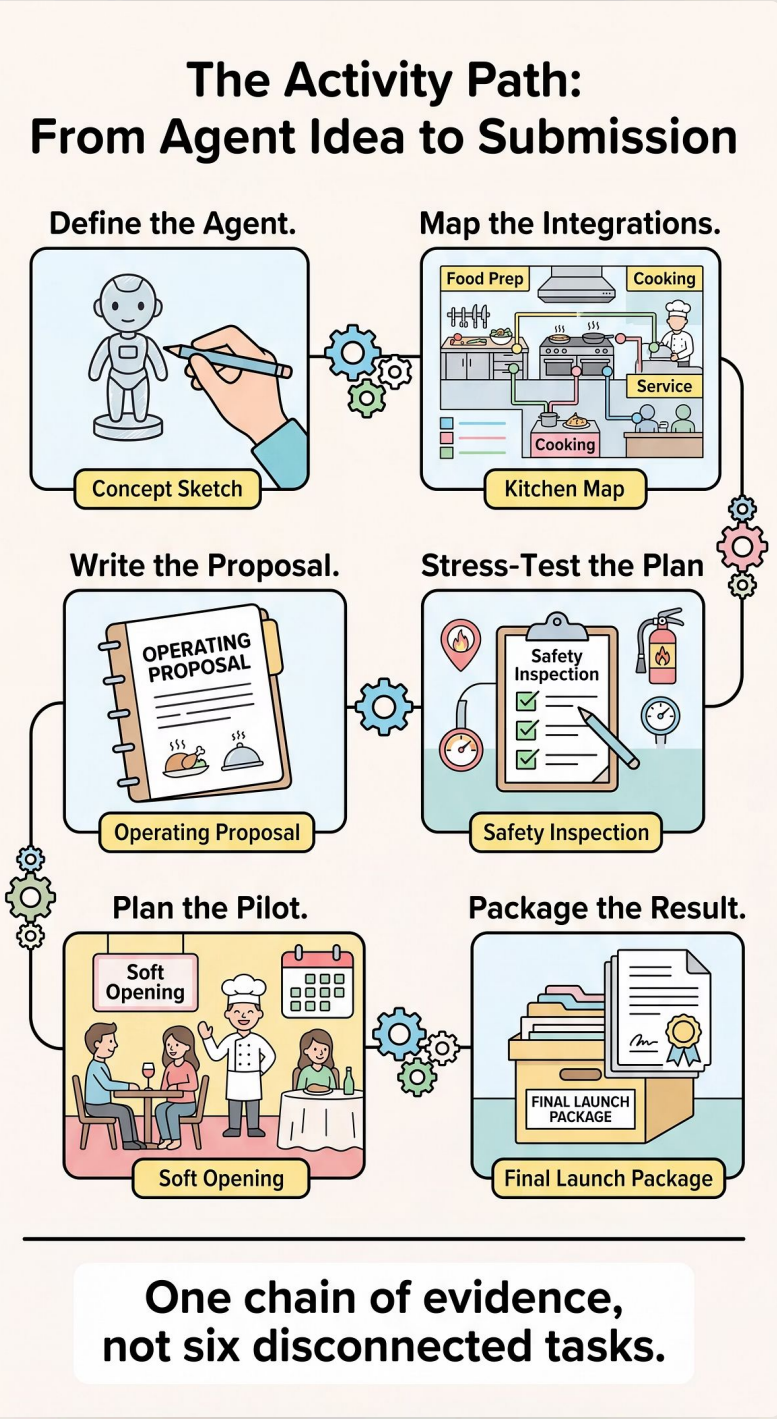
05

Build and Check the Microsite

Prompts 5 and 6: Write the leadership email and prepare the website handoff.

One chain of evidence, not six disconnected tasks.

Metaphor (restaurant): concept → kitchen map → safety check → soft opening → launch package



Pre-Demo

The Proposal Blueprint (What Leaders Need to See)

Agent type + where it lives
Slack/Email/Web/Helpdesk

Workflow supported
Step-by-step, with handoffs

Integration plan
Systems, data, and barriers

Safety + ethics
HITL, guardrails, monitoring, fallback

Communication style
Neutral vs empathy; escalation triggers

Success definition
3 KPIs + pilot plan + iteration loop

Metaphor (restaurant): concept → floor plan → food safety → staffing → metrics



Pre-Demo

Data Compatibility & Governance (LLM-Ready Inputs)

Most integration pain is data, not models.

Expect Mismatches

Data comes in varied forms; anticipate discrepancies like "Cust_ID" vs "CustomerNumber", missing fields, and inconsistent formats across systems.

Plan the Transform

Establish a clear process to extract, clean, normalize, and structure data, making it ready for LLM prompts and specific tools.

Add Quality Checks

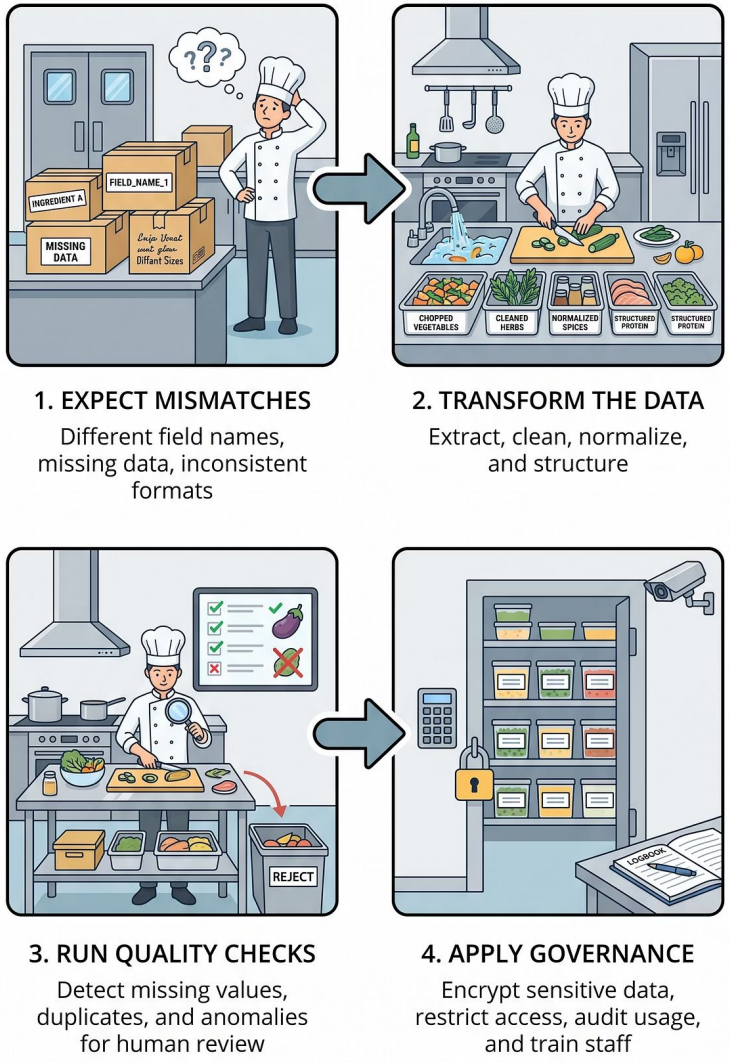
Implement robust checks for missing data, duplicates, or unusual formats. Flag anomalies for human review to maintain data integrity.

Ensure Governance

Encrypt sensitive data, restrict access, audit usage, and train staff on responsible data handling to maintain compliance and trust.

In your assignment: Write 3 "data requirements" and 3 "data risks" (with mitigation strategies for each).

DATA COMPATIBILITY AND GOVERNANCE



1. EXPECT MISMATCHES
Different field names, missing data, inconsistent formats

2. TRANSFORM THE DATA
Extract, clean, normalize, and structure

3. RUN QUALITY CHECKS
Detect missing values, duplicates, and anomalies for human review

4. APPLY GOVERNANCE
Encrypt sensitive data, restrict access, audit usage, and train staff

ASSIGNMENT OUTPUT: 3 DATA REQUIREMENTS + 3 DATA RISKS WITH MITIGATIONS

Pre-Demo

Performance Planning: Latency, Load, and Fallbacks

If it's slow or flaky, users won't trust it.

Identify Latency-Sensitive Steps

Pinpoint where real-time chat, live routing, or critical alerts demand immediate responses from the agent.

Local vs. Cloud Execution

Determine which agent components run locally for speed and data sensitivity, versus those in the cloud for scalability and access.

Monitor Key Metrics

Track inference latency, error rates, system load, and tool-call failures to proactively manage performance bottlenecks.

Implement Robust Fallbacks

Design strategies to queue requests, gracefully degrade service, or switch to smaller backup models when primary systems falter.

In your proposal: Include 2 performance risks + 2 "what we do when it fails" rules to demonstrate resilience.

"Performance Planning: Latency, Load, and Fallbacks."

1. Identify Time-Critical Steps

Live chat, routing, and urgent alerts

Restaurant metaphor



Rush-hour orders

2. Choose Local or Cloud Execution

Balance speed, privacy, scalability, and access

Restaurant metaphor



Kitchen capacity

3. Monitor Key Metrics

Latency, error rate, system load, and tool failures

Restaurant metaphor



Service monitoring

4. Activate Fallbacks

Queue requests, reduce service, switch models, or escalate to a human

Restaurant metaphor



Backup stations and manager intervention

"Proposal output: 2 performance risks + 2 failure-response rules."

Agent Performance & Reliability: Examples

Identify Latency-Sensitive Steps

- Live chat: Agent responds to "Where is my order?" in <2s
- Fraud alert: Suspicious login triggers immediate alert & lock
- Call assist: AI suggests answers in real-time during support calls

Local vs. Cloud Execution

- Local: Classify incoming tickets for speed & privacy
- Cloud: Generate responses using knowledge base & LLM
- Hybrid: Local detects urgency, Cloud writes full reply

Monitor Key Metrics

- Latency: Average response time is 1.8 seconds
- Error rate: Percent of misrouted tickets per day
- Tool failure: Failed Zendesk API call triggers an alert

Implement Robust Fallbacks

- Queue: Overloaded requests held, users see delay message
- Backup model: Primary model fails, smaller model takes over
- Human escalation: Low-confidence tickets routed to a person

Agent Performance and Reliability: Examples.

LATENCY-SENSITIVE TASKS



RUSH HOUR:

- Live chat under 2 seconds
- Immediate fraud alerts
- Real-time call assistance

LOCAL, CLOUD, OR HYBRID



Local for speed and privacy

Cloud for generation and scale

Hybrid for urgency detection plus full responses

KEY METRICS



- Response time
- Misrouting rate
- Tool-call failures

FALLBACKS



- Queue overloaded requests
- Switch to a backup model
- Escalate low-confidence cases to a human

Workflow Spec: Make It Defensible in One Page

Start with the trigger

"What event starts the agent?"

Then list steps as

Classify → prioritize → route → draft → log → escalate

Add a confidence score

So humans know when to trust it

Define "will do" vs "will not do"

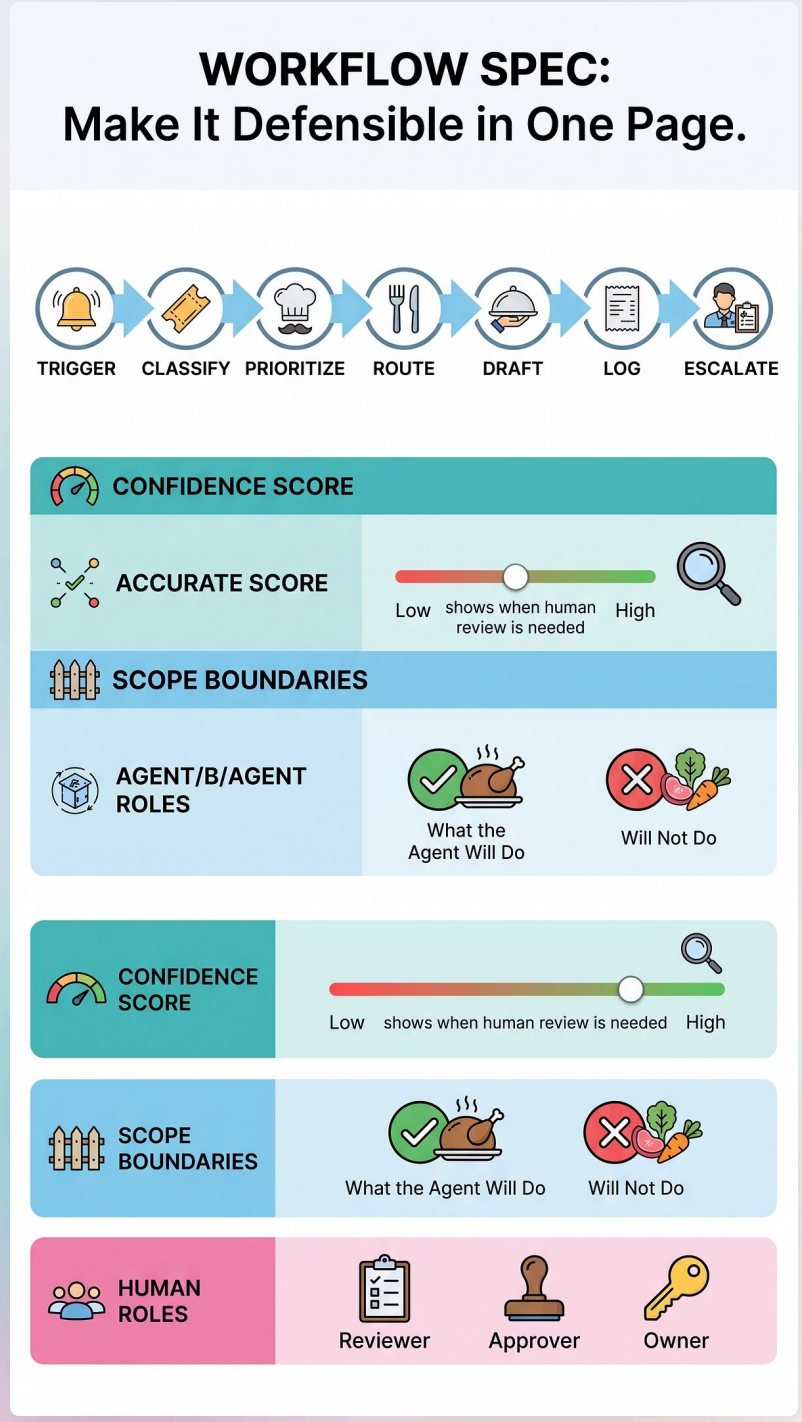
Scope boundaries prevent accidents

Default to "draft/suggest" responses; require approval to send/commit

Name the human roles

Reviewer, approver, owner

Metaphor (restaurant): ticket comes in → station assignment → expo checks → manager steps in



Integration Map: Where the Agent Reads and Writes

Systems to connect (typical)

Helpdesk, email, CRM, knowledge base, analytics

Data needed

Ticket text, customer context, policy/articles, history, priority signals

Access model

Least privilege (start read-only; add write access later)

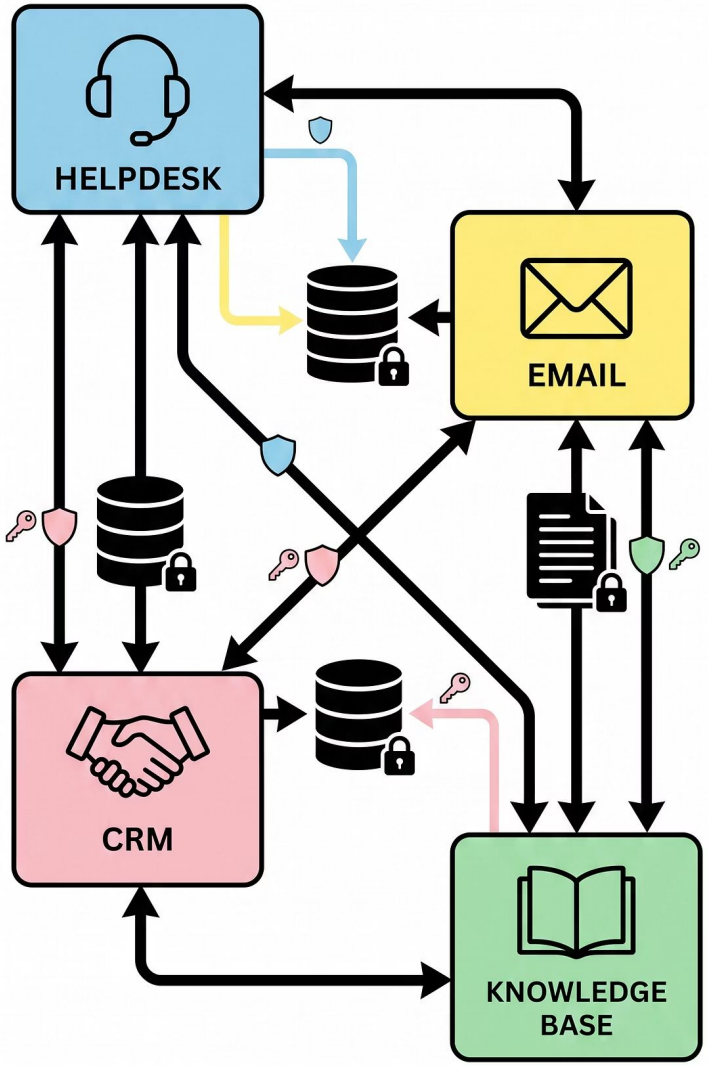
Barriers you must name

- Siloed data
- Inconsistent fields
- Latency
- Audit needs

Build for reality: outages happen; permissions fail; data is messy

"Integration" means: the agent can safely use your apps with permission

Metaphor (restaurant): POS → kitchen printer → inventory → receipts must sync



Tools and Meta-Tools: What Does Each Part Do?

Applications are where work happens. Meta-tools coordinate work across them.

Business Applications

Slack and Discord support communication. Notion organizes information and tasks.

Automation Platforms

Zapier and n8n coordinate actions across applications.

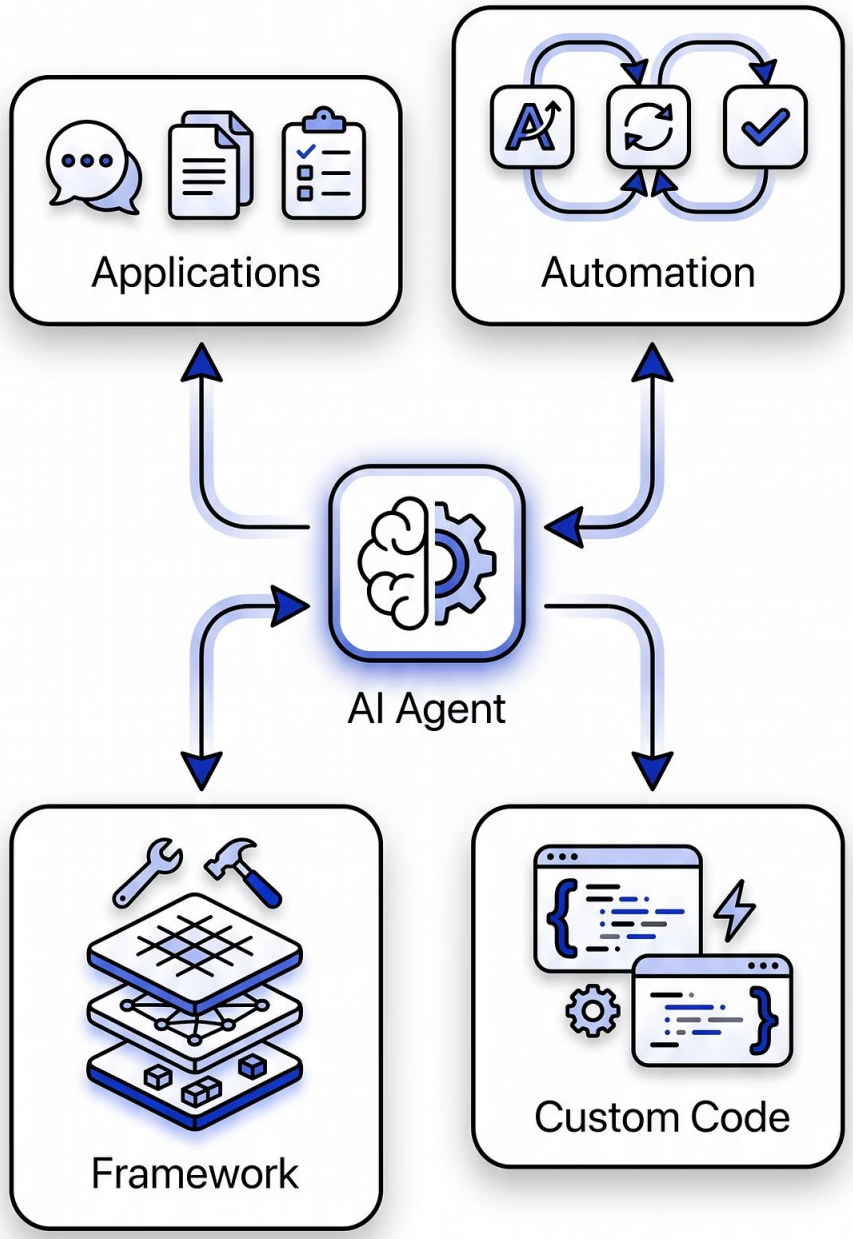
Agent-Building Framework

LangChain gives developers building blocks for multi-step agents with memory and tool use.

Custom Programming

Python supplies organization-specific logic when existing tools do not fit.

Metaphor (restaurant): applications are workstations; automation coordinates service; custom code handles special requirements.



System Audit Before the Agent Gets Access

Map the real environment before choosing the connector.

Inventory the Systems

List the helpdesk, email, CRM, knowledge base, analytics tools, and system owners.

Trace the Data

Document where the data lives, its format, quality, sensitivity, and source of truth.

Test the Access

Check available APIs, permissions, rate limits, dependencies, and likely failure points.

Choose the Bridge

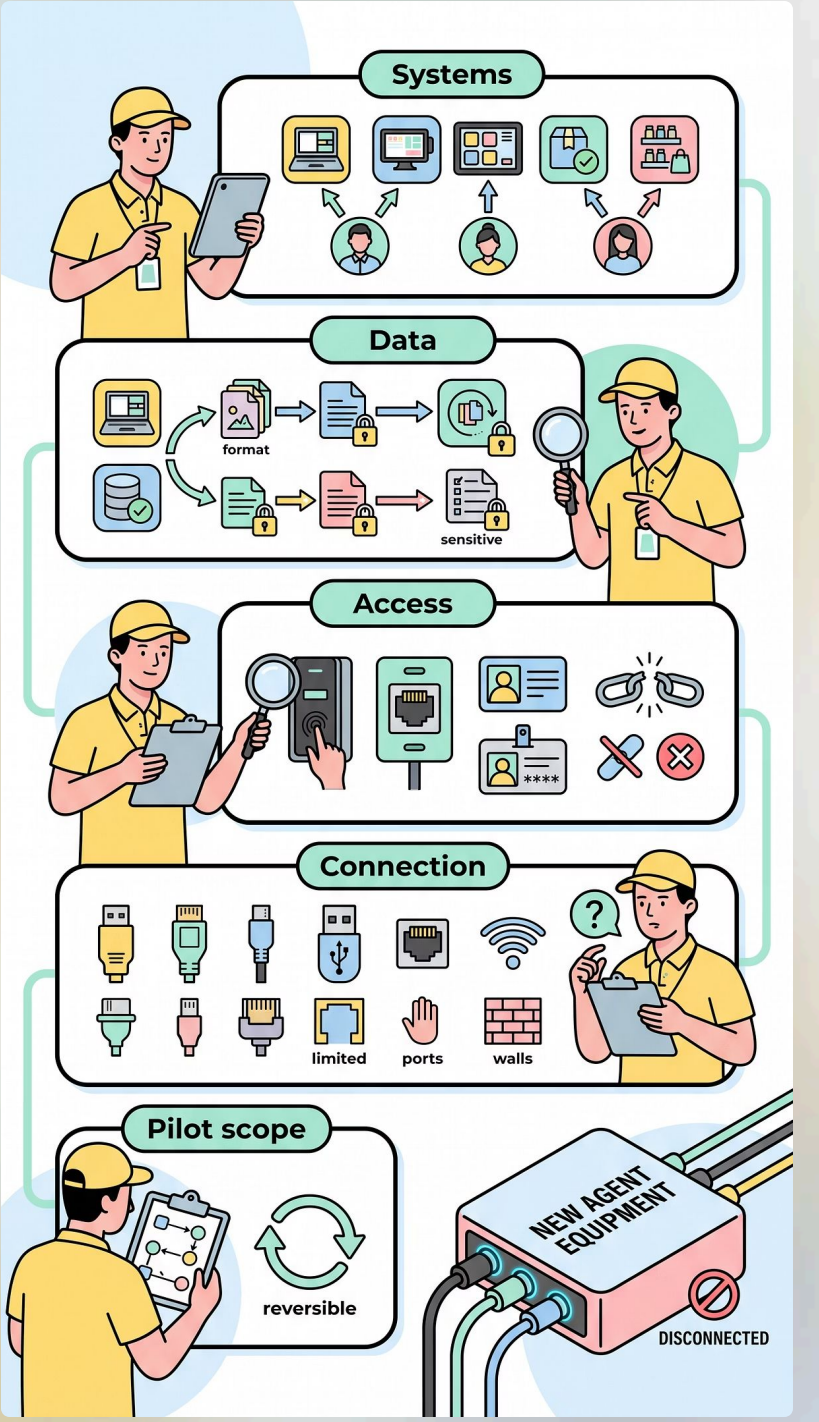
Select a direct connector, middleware layer, or small microservice based on the constraints.

Select the First Scope

Begin with work that is routine, reversible, measurable, and lower risk.

Audit output: system map + data risks + named owners + fallback path

> Metaphor: inspect the kitchen before installing new equipment.



Legacy Integration: Connect Without Rebuilding Everything

Modernize the connection without replacing every system.

Legacy Constraint

Older "all-in-one" applications tightly bundle screens, business rules, and data. Changing one part can disrupt others.

Middleware

Software between old and new systems. It translates data, routes requests, and enforces security rules.

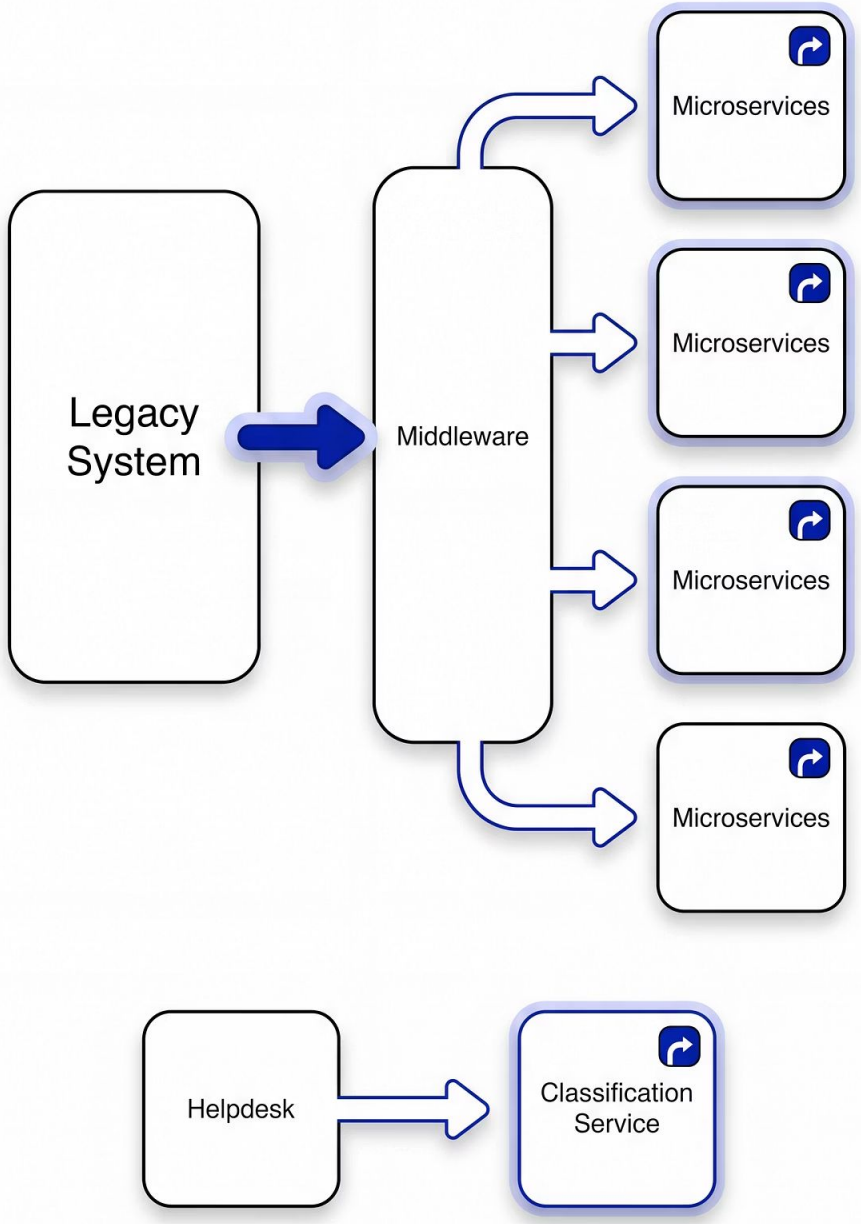
Microservices

Small, independent services that each handle one task. Each function can be updated or scaled separately.

Illustrative Example

A separate classification service can support the existing helpdesk rather than replace it.

Metaphor (restaurant): add a dedicated prep station without rebuilding the whole kitchen.



Two Integration Paths: Standard Tool Layer vs Custom APIs

Path A: standardized tool layer (MCP-style)

Best when you want portability + consistency across tools

Path B: custom APIs/webhooks/connectors

Best when you need tight control or have unique legacy constraints

---

Tradeoffs to state

Speed, maintenance, portability, security, observability

Rule of thumb

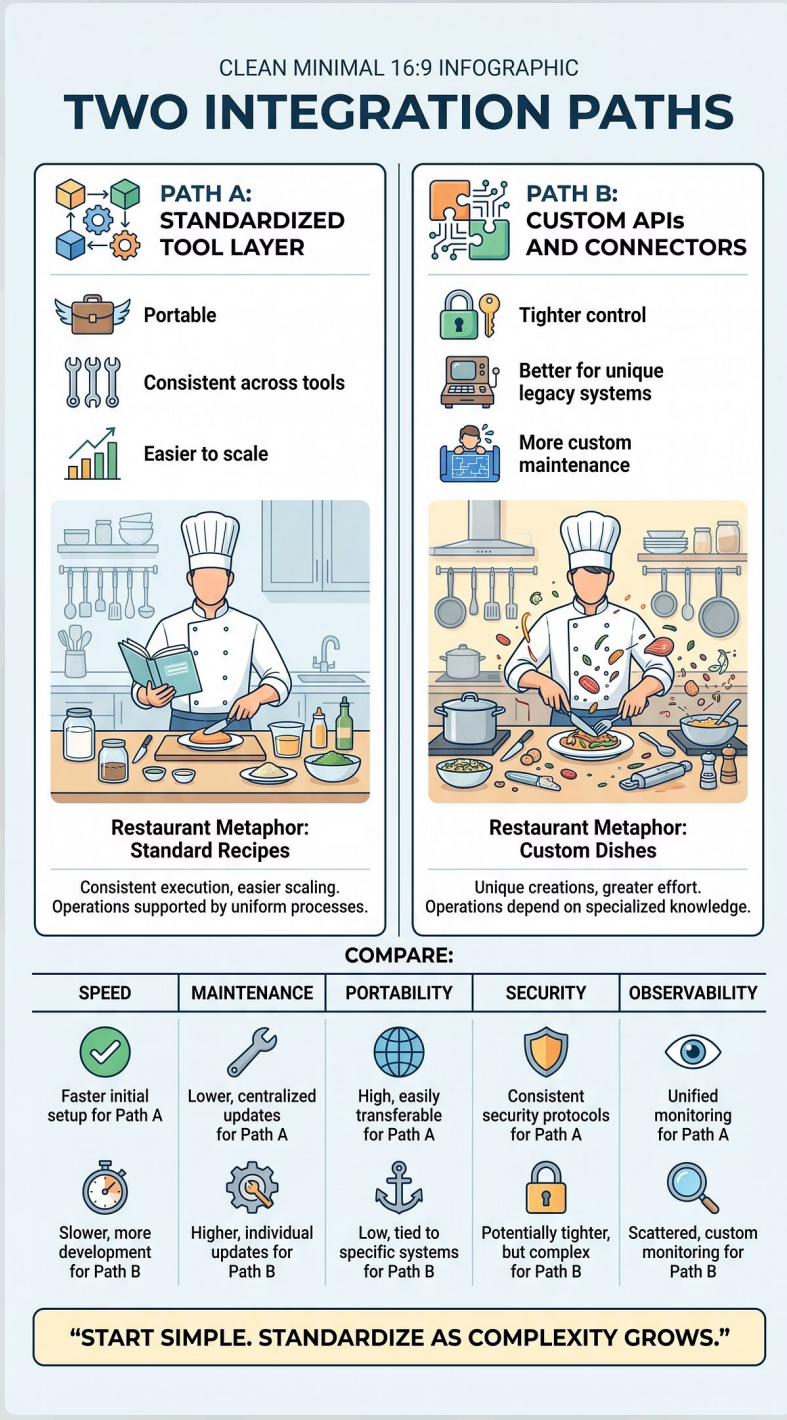
Start simple; standardize when scale/complexity grows

Always document

What tools can be called, with what permissions, and when

Don't oversell: your proposal should acknowledge constraints explicitly

> **Metaphor (restaurant):** standard recipes vs custom dishes—both can work, but ops must support them



MCP in Plain English: From Intent to Action

A standardized tool connection can still use an application's API.

MCP: Model Context Protocol

A shared standard connecting AI agents to external tools and data.

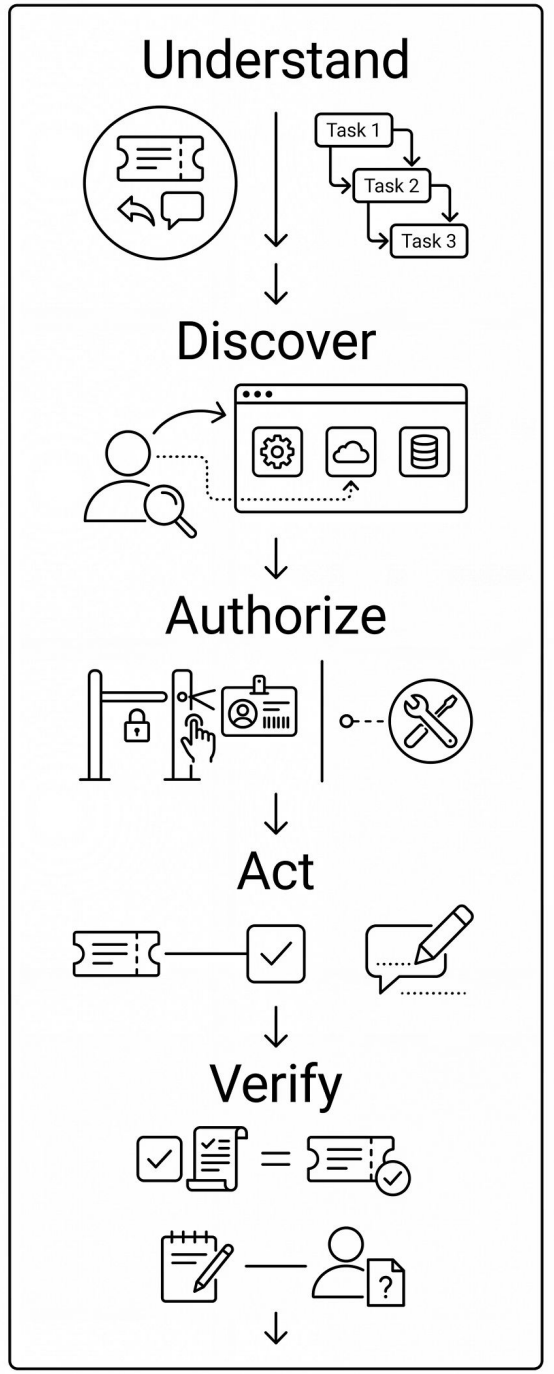
API: Application Programming Interface

A defined way for software to request data or actions from another system.

In the module's Spotify example, the agent matches songs using APIs or catalog matching, then creates a playlist with authenticated access.

Key distinction: MCP and APIs are not necessarily competing alternatives. They can support the same workflow.

Metaphor (restaurant): one service window coordinates requests to several kitchen stations.



MCP in Plain English: From Intent to Action

MCP-style integration gives agents a consistent, governed way to use approved tools.

User Intent

"Route this urgent ticket and draft a reply."

Understand

Convert the request into a structured sequence of steps.

Discover

Identify the approved tools and actions available for the task.

Authorize

Check the user identity, permissions, scope, and approval requirements.

Act

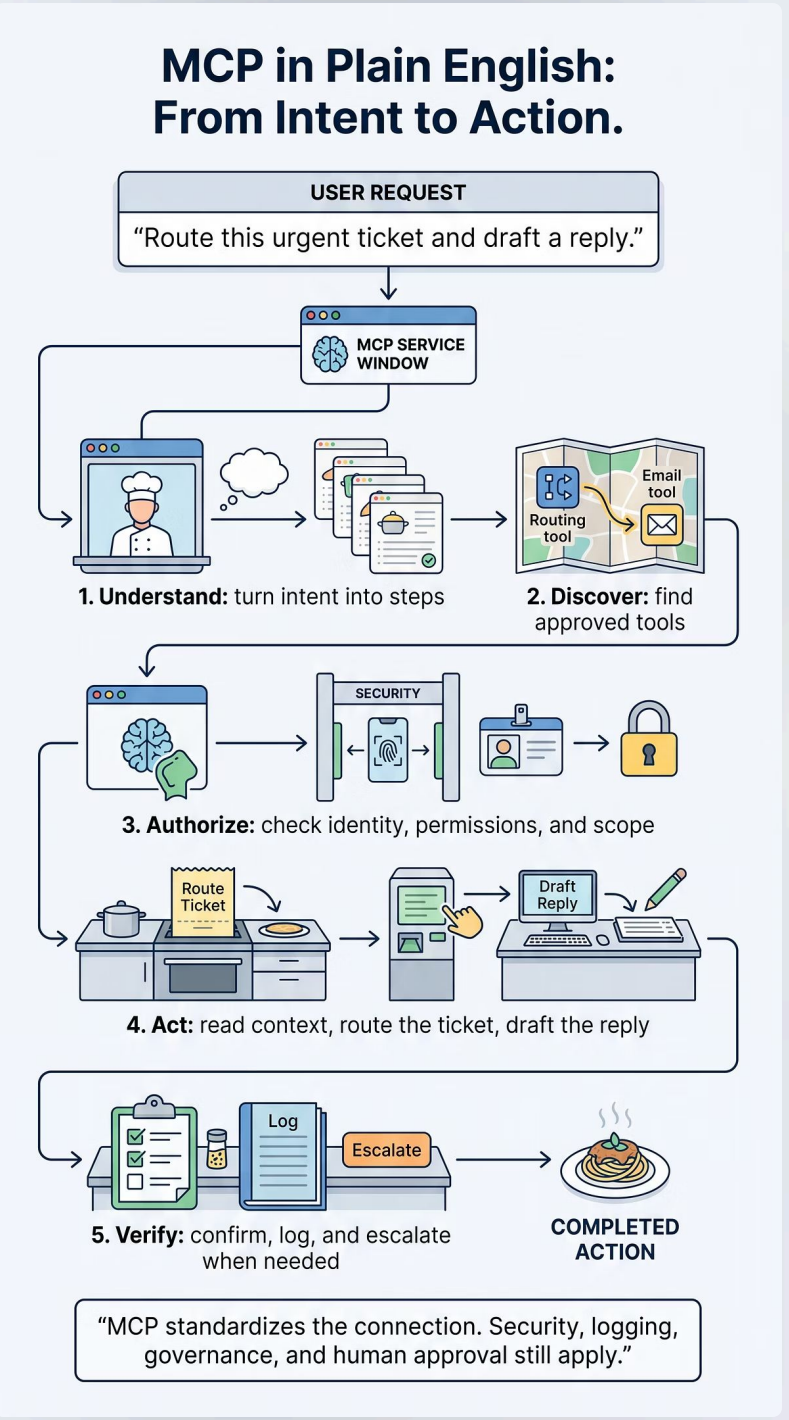
Read the ticket context, check relevant systems, route the request, and draft the reply.

Verify

Confirm the result, log the action, and escalate when human approval is required.

MCP standardizes the connection. It does not remove security, logging, governance, or human approval.

> **Metaphor:** one service window, many stations, each with its own permission rules.



Safety Rules: How We Prevent "Agent Accidents"

Core rule: no irreversible actions without explicit confirmation

Guardrails

Allowlist tools, role-based access, rate limits, timeouts

Quality controls

Confidence thresholds + safe fallback when uncertain

Data safety

Redact sensitive info, avoid over-collection, log access

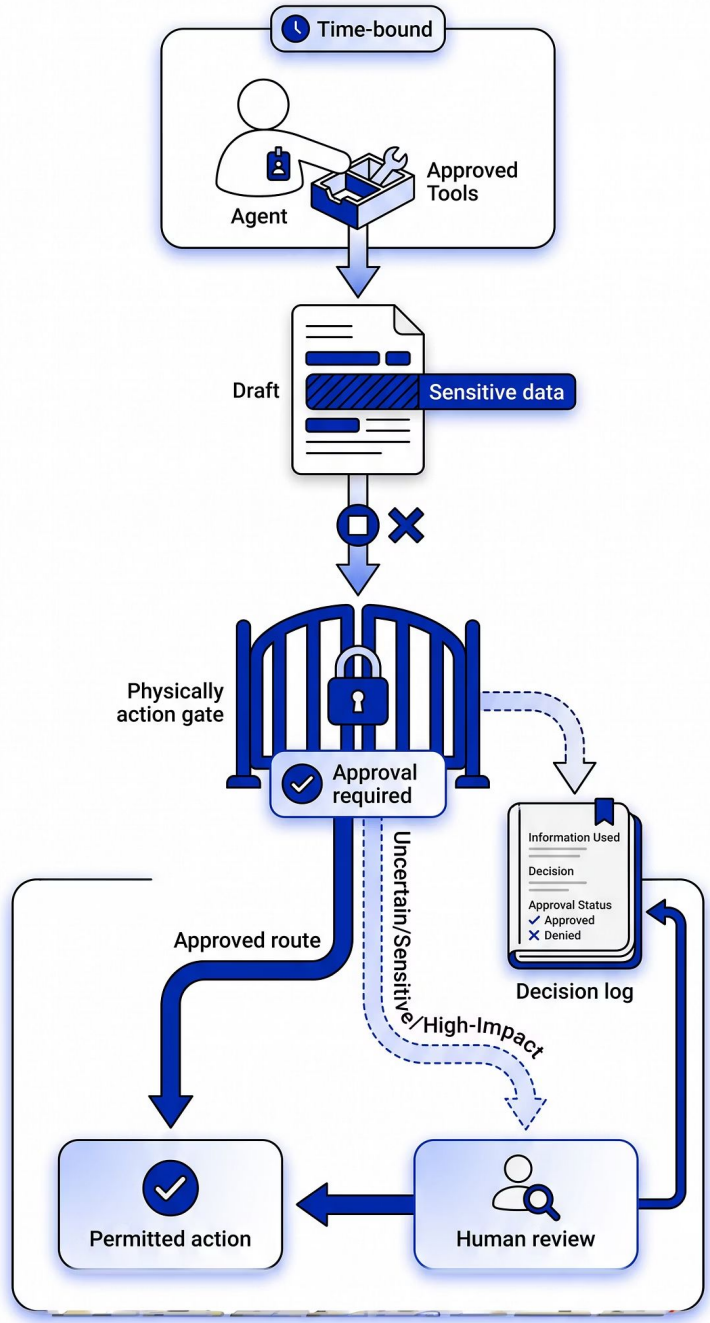
Decision logs

What it saw, what it did, why it did it

Escalation triggers

Sensitive topics, low confidence, high impact, frustration

Metaphor (restaurant): allergy checks + temp logs + manager approval for high-risk orders



Edge Cases: The 8 Failures You Must Pre-Plan

1

Misrouting (wrong team)

Detect via overrides + feedback loops

2

Sensitive data exposure

Detect via classifiers + redaction rules

3

Angry users

Detect sentiment + escalate quickly

4

Wrong policy guidance

Require citations + human review for policy claims

1

Conflicting tickets

Dedupe + link + route to owner

2

Tool outages

Circuit breakers + queue + "degraded mode"

3

Permission failures

Fail closed + log + request access workflow

4

Low-confidence classification

Human review, not autopilot

Metaphor (restaurant): rush hour + outages happen—your process must degrade gracefully

Empathy Is a Control Point, Not Just Tone

The agent should recognize emotional risk and choose the safer next action.

Detect

Look for frustration, urgency, repeated contact, sensitive topics, and changing sentiment.

Acknowledge

Recognize the concern clearly without pretending to feel emotion or promising an outcome.

Act

Explain the next step, available option, expected timeline, or information still needed.

Escalate

Transfer to a human when emotion, sensitivity, impact, or uncertainty exceeds the agent's authority.

Operational Empathy

Instead of:

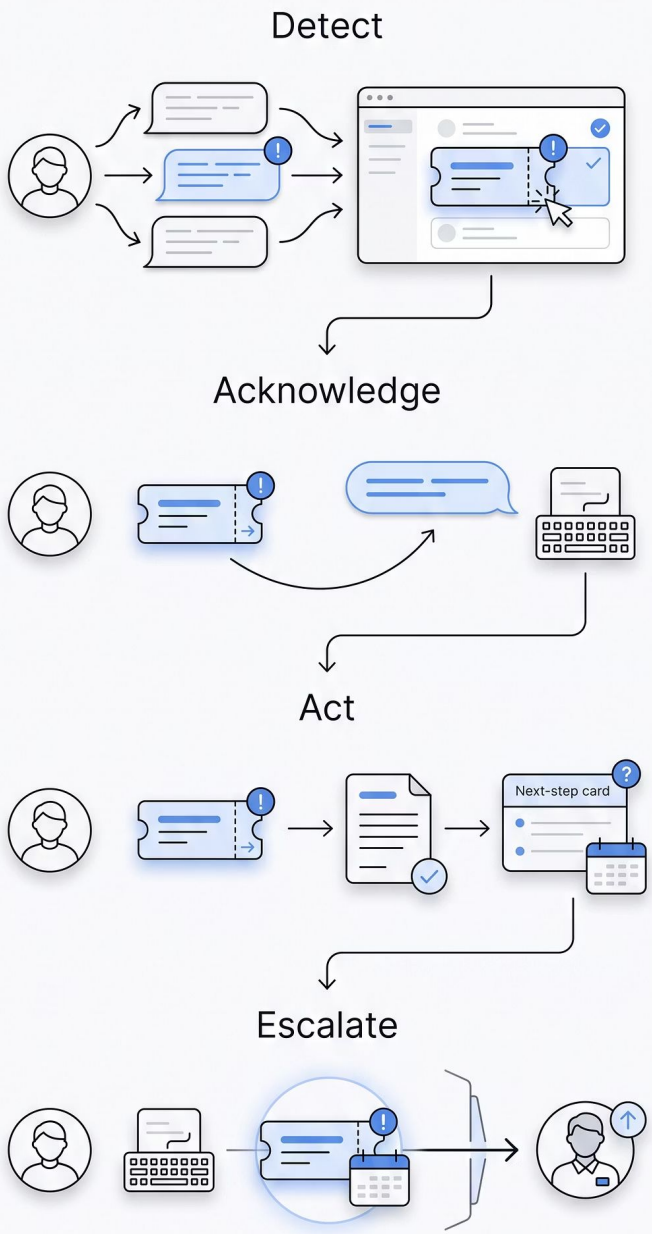
"Sorry for the inconvenience."

Use:

"I understand why this is frustrating. I will verify the policy, explain the next step, and bring in a human if approval is required."

⚠ **Do not trade accuracy for warmth. Never invent policy, compensation, or outcomes.**

> **Metaphor:** a strong server listens, confirms, acts, and calls the manager when the stakes rise.



Fine-Tuning: Learn From Examples and Instructions

Fine-tuning enhances an existing language model by training it on specific data for a particular task or business need.

Supervised Fine-Tuning (SFT)

Learning from labeled examples, such as support tickets categorized by topic or urgency.

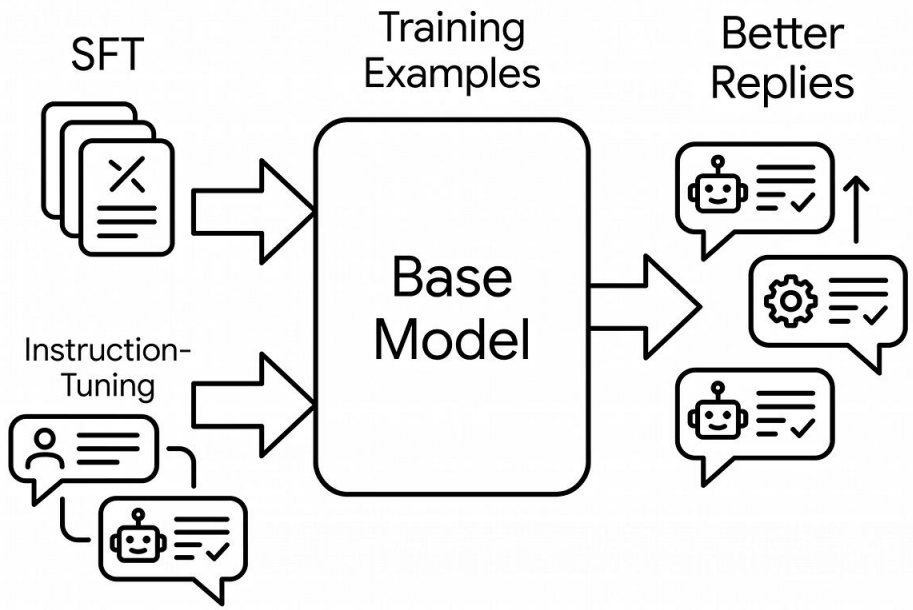
Instruction-Tuning

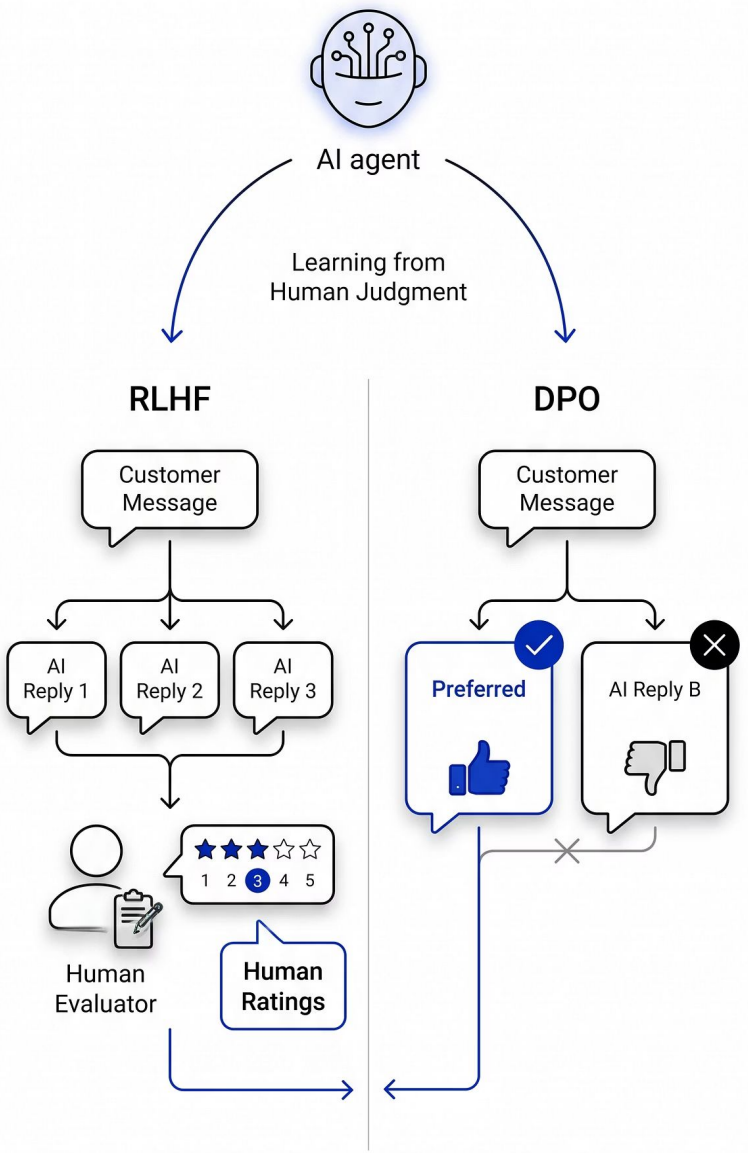
Training on examples that demonstrate how to follow specific instructions, like acknowledging frustration in a customer interaction.

Application: Ticket Triage

Combined training examples show both how to classify a request and how to craft an appropriate, empathetic reply.

Metaphor (restaurant): train an experienced server on your restaurant's specific orders and service standards.





Learning From Human Feedback: RLHF and DPO

Human judgments can guide training toward more helpful replies.

Reinforcement learning from human feedback (RLHF)

Human evaluators rate generated responses. Their feedback guides training toward helpful, policy-aligned behavior.

Direct preference optimization (DPO)

Train on pairs: a preferred response and a rejected response to the same request.

Illustrative ticket example

Prefer a reply that acknowledges frustration and explains the next step over a vague, repetitive apology.

> **Metaphor (restaurant):** coaching uses service ratings and examples of stronger versus weaker responses.

PEFT: Adapt a Small Part of the Model

Parameter-efficient fine-tuning (PEFT) targets a small part of the model.

What changes

Update selected parameters, the model's learned settings, rather than the whole model.

Why it matters

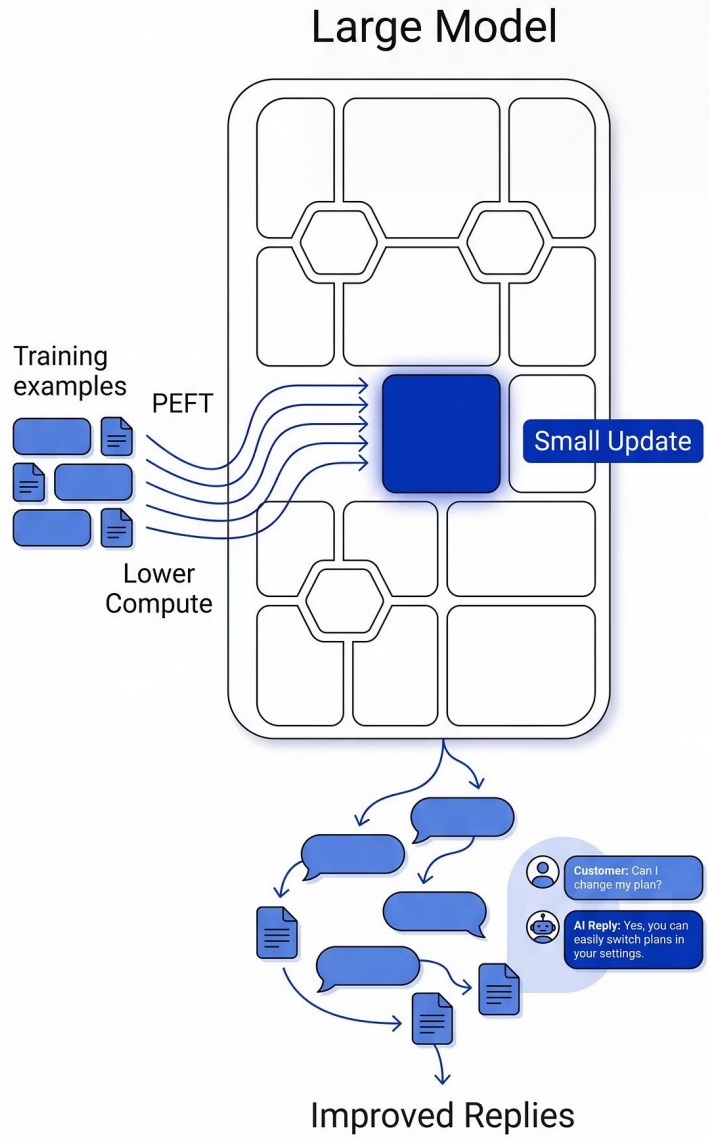
Focused training updates can reduce computing needs and make revisions easier.

Module example

A telecom team adds a small trainable component called an adapter to improve how it explains new plans empathetically.

Test that replies remain accurate and appropriate.

> **Metaphor (restaurant):** teach a new menu item without repeating the entire staff training.



What Goes Into Empathy-Training Data?

Empathy training isn't just about apologizing; it's about learning from rich, human-labeled conversations to develop nuanced responses.

Use support conversations with human-created labels.

Emotional Lexicon

Examples of language that **acknowledges frustration** or concern, capturing the depth of customer emotions.

Cultural & Demographic Sensitivity

Examples spanning diverse dialects, cultural norms, and different expectations of formality across various demographics.

Contextual Nuance

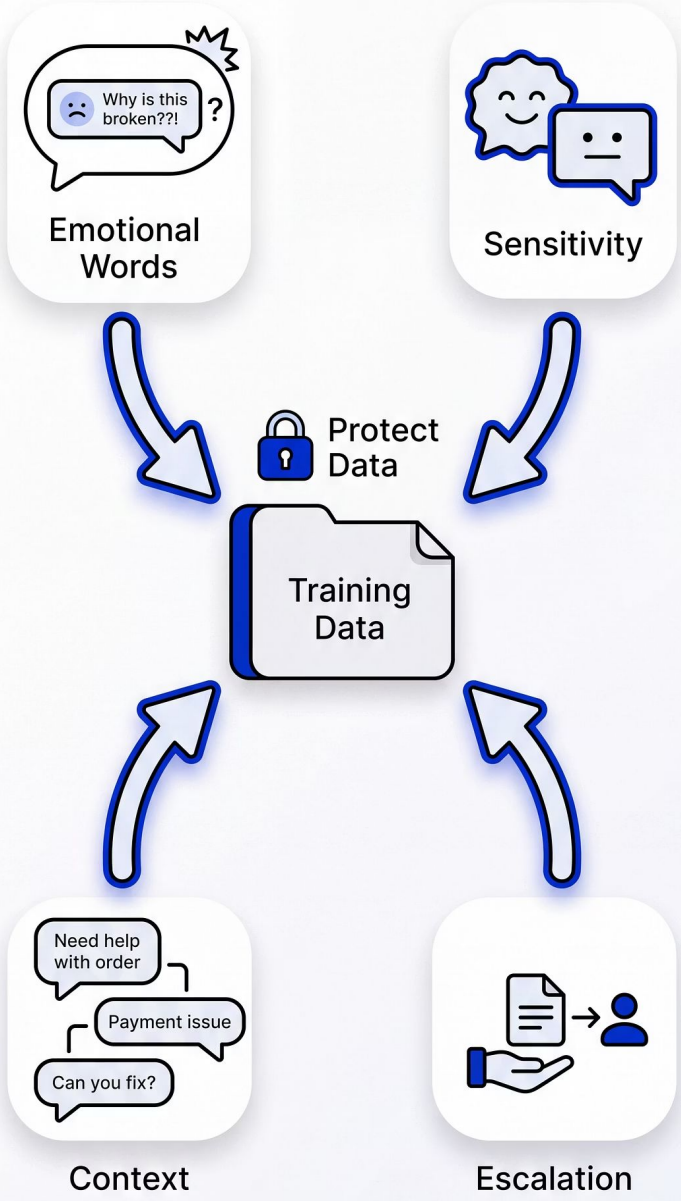
Multi-turn conversations that illustrate precisely **when to pivot from reassurance to practical action**, based on evolving dialogue.

Escalation Triggers

Situations explicitly labeled as requiring human intervention, with detailed sentiment and urgency metrics recorded for each case.

⚠ **Protect the Source Data:** Anonymize customer transcripts and implement strict access controls before using them for training.

> **Metaphor (restaurant):** Rehearse a variety of guest situations, not just a single apology script.



Measurement: Prove Value, Don't Just Demo

Test empathy without losing accuracy. A caring tone is not enough; critical evaluation ensures genuine progress.

Always check replies on conversations excluded from training data to validate performance in unseen scenarios.

Response Fit

Overfitting can produce rehearsed, unnatural scripts. Underfitting can leave responses insensitive to the situation or context.

Knowledge Retention

Guard against catastrophic forgetting, where new empathy training weakens earlier knowledge. Mix empathy examples with product-knowledge tasks to maintain balance.

Channel Consistency

Ensure empathy and accuracy are consistent across all communication channels (chat, email, voice) by using shared guidelines and human audits.

Human Fallback

Implement clear escalation paths. The agent must seamlessly transfer to a human when emotional cues are misunderstood or exceed its capabilities.

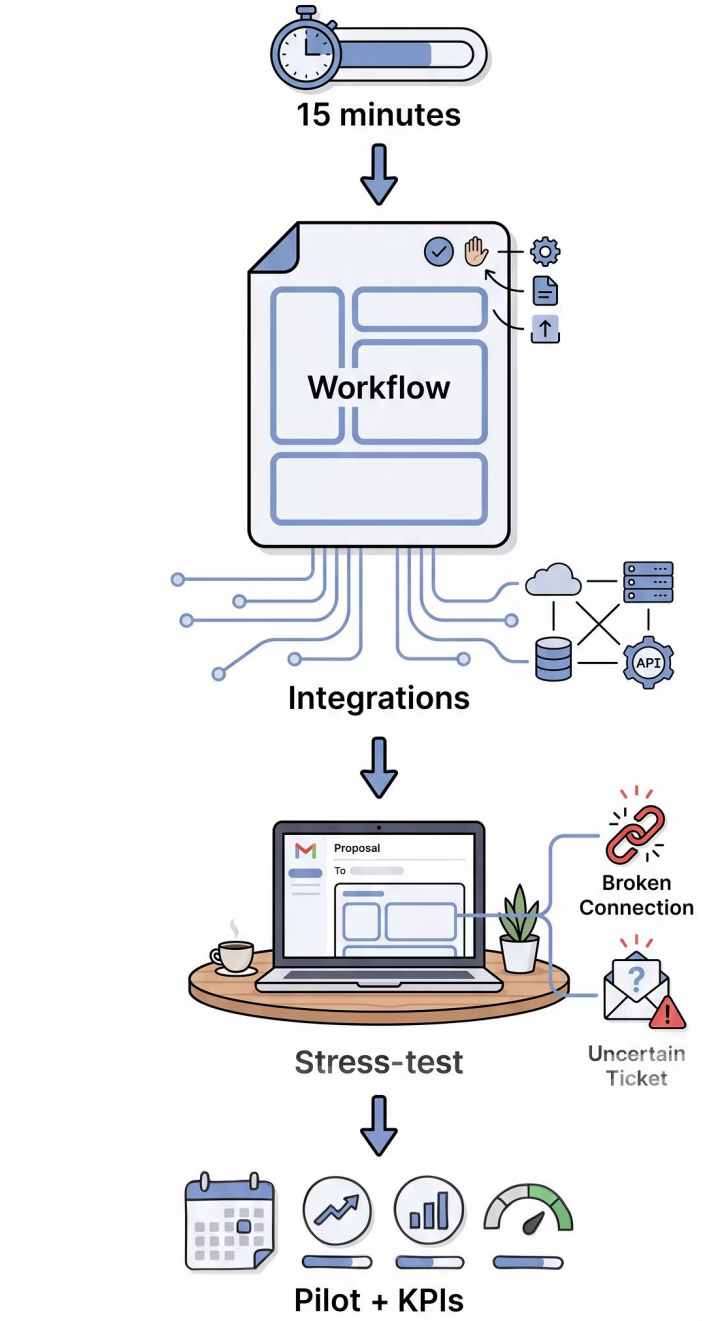
Metaphor (restaurant): test courtesy, menu knowledge, and judgment before serving guests at scale.

Transition

Live Demo: From Billing Complaints to a Pilot Decision

- We will work through HarborLink Mobile, a fictional telecom company facing billing complaints after a plan migration.
- Build the operating plan Define the assistant's workflow, system access, and human responsibilities.
- Test the difficult case Introduce conflicting policies, repeated transfers, and an unavailable billing lookup.
- Prepare the leadership decision Create the pilot plan, final recommendation, and interactive review microsite.

> **Metaphor (restaurant):** inspect the kitchen and rehearse the service before opening.



What You Will See: One Connected Business Case

Our demo will unfold a comprehensive approach, from strategic planning to a data-driven recommendation, showcasing every phase of a deployable agent solution.

Operating Plan

Clearly define the assistant's preparation tasks and human responsibilities, ensuring seamless collaboration.

System Map

Visualize required information flows, proposed system access, and identify any unresolved technical constraints.

Incident Stress Test

Observe how conflicting data or unexpected scenarios impact the plan and how the agent adapts or flags for human review.

Pilot Charter

Establish clear ownership, three specific performance targets, review requirements, and crucial "stop rules" for safe deployment.

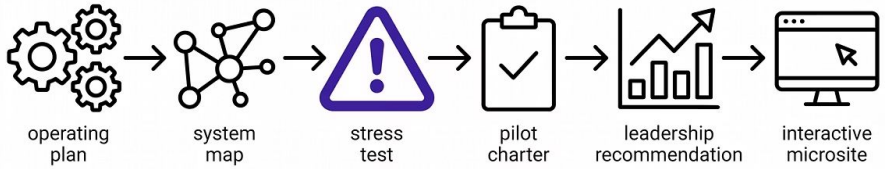
Leadership Recommendation

Formulate a data-backed recommendation, presenting evidence rather than a preconceived outcome.

Interactive Microsite

Explore the detailed demo: review tickets, system integrations, approval workflows, and access a downloadable proposal.

> **Metaphor (restaurant):** one opening plan, from kitchen inspection to management review.



The HarborLink Case: Billing Support After a Plan Migration

The problem: Customers question unexpected charges and repeat their stories across support teams.

The input

Billing-related tickets received through email and web chat.

The assistant's job

Classify requests, recommend the right team, and draft a response.

The human's job

Review recommendations, move tickets, and send customer messages.

The boundary

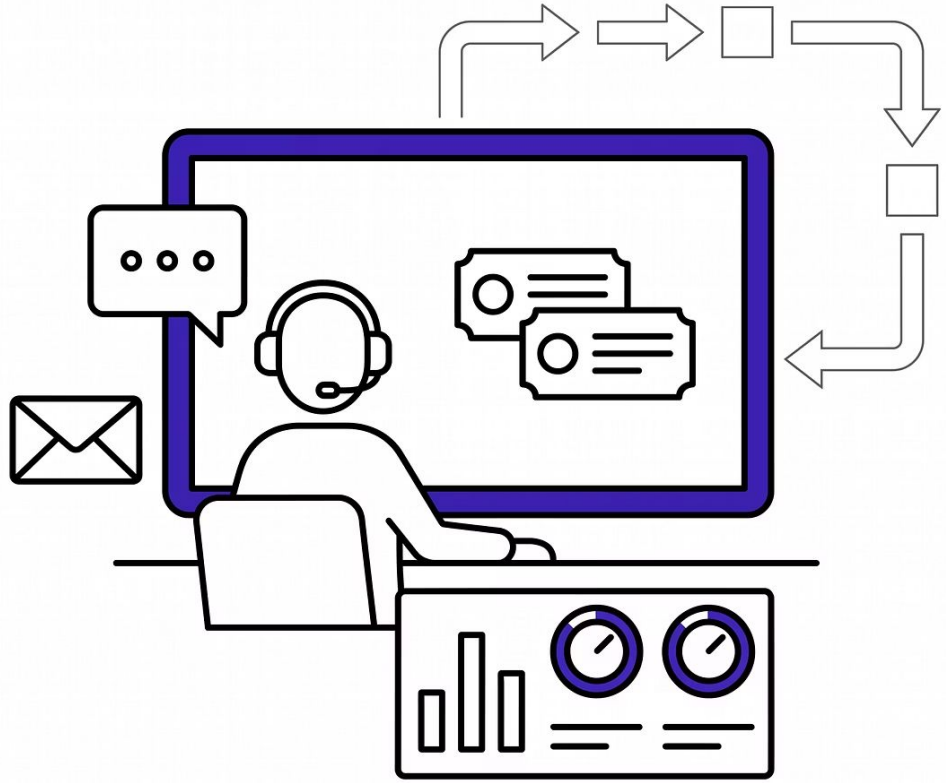
The agent cannot issue credits, change plans, or settle disputes during this pilot.

The goal

Faster first responses, fewer reassignments, and fewer repeat contacts.

Fictional company and evidence. Proposed benefits, not measured results.

> **Metaphor (restaurant):** suggest the right station; the responsible person handles the order.



What the Agent Actually Does (A Workflow You Can Defend)

This is the agent loop in plain English: trigger → decide → act → verify → escalate.

Trigger

A new ticket arrives

Decide

Classify intent, urgency, sensitivity, and uncertainty. Confidence is not measured in this demonstration.

Act

Route + draft a suggested response (not auto-send by default)

Verify

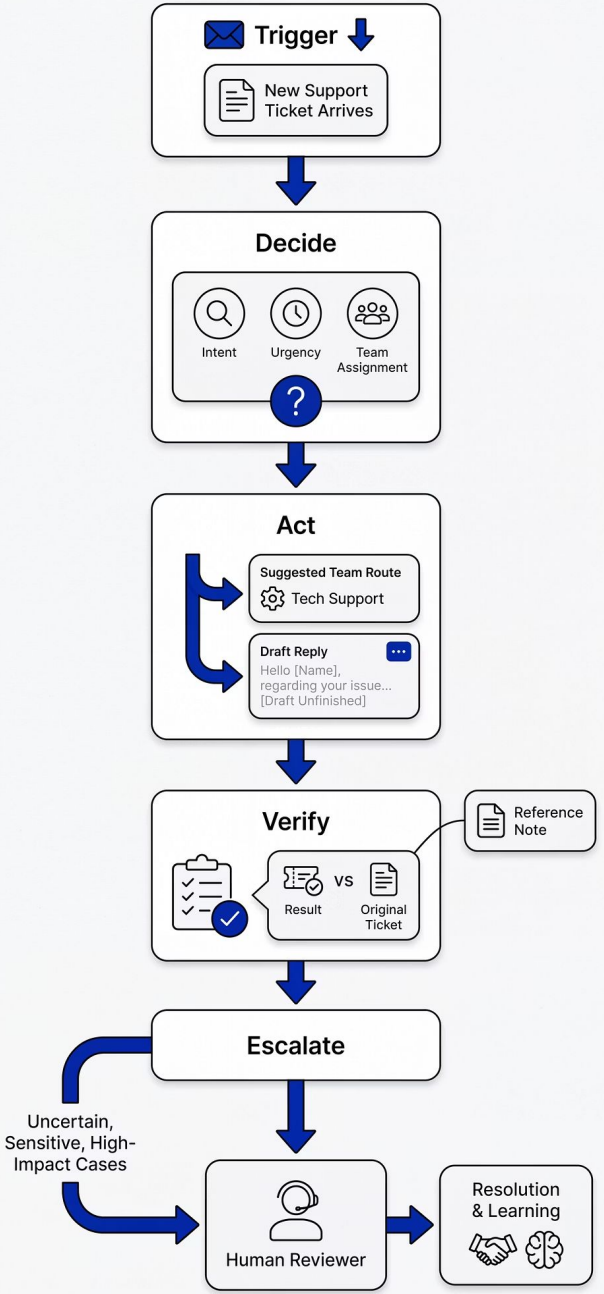
Check the supporting evidence. Separate proposed actions from confirmed results and flag what still needs verification.

Escalate

Low confidence, sensitive topic, or high impact → human review

Non-technical: it follows a checklist so nothing slips through the cracks

Metaphor (restaurant): the host seats, the server confirms, the kitchen prepares, the manager steps in when it's tricky



Integration Plan (Module 3: Real Systems, Real Constraints)

Agents only create value when they connect to the tools your org already runs.

Systems to connect → Proposed system environment

Zendesk, Salesforce, legacy billing, SharePoint, email/web chat, and Power BI.

Data needed

Ticket text + context + relevant policies/articles

Two integration paths → Choose the connection for each system

A standardized tool layer can use APIs underneath. Select the proposed connection based on access, data quality, and safe failure.

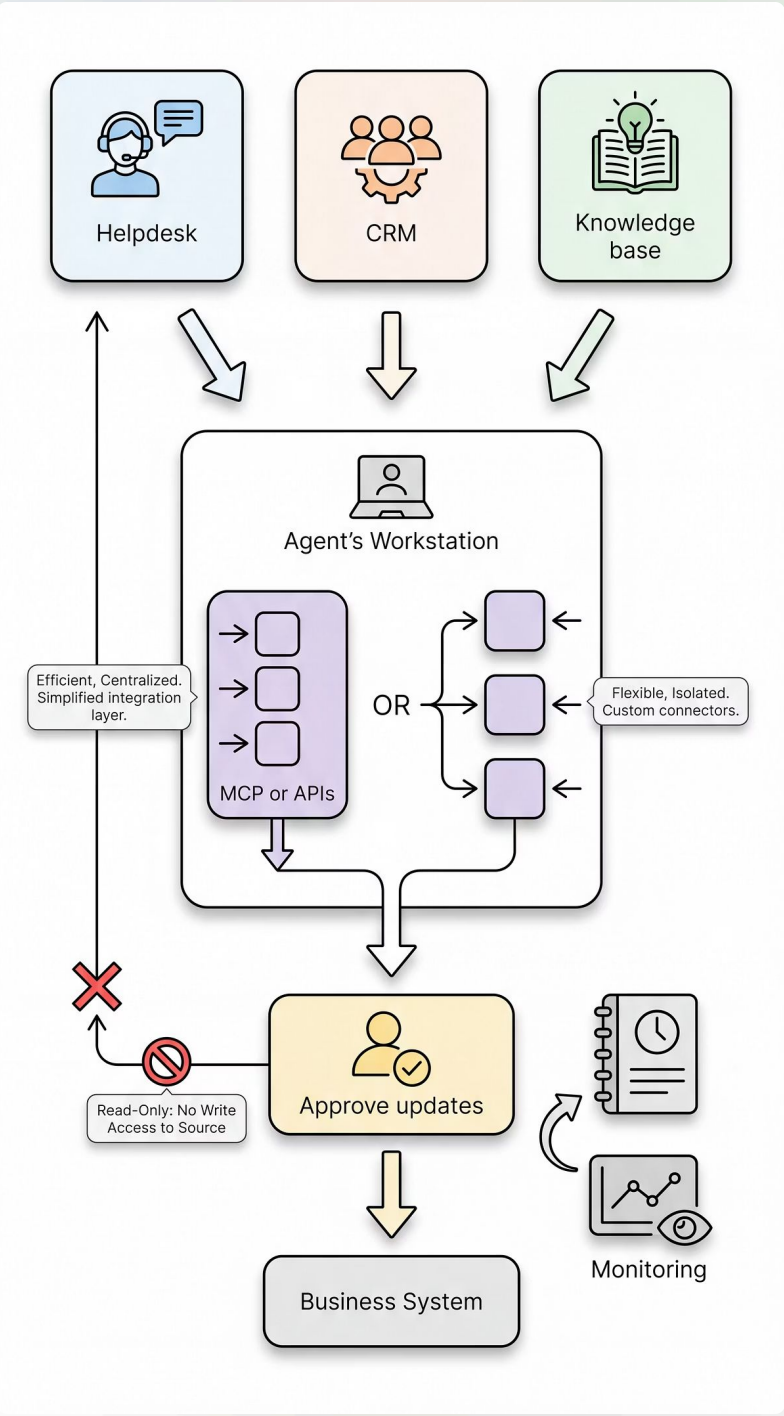
Expected barriers

- Permissions
- Siloed data
- Inconsistent fields
- Latency
- Audit needs

Keep it safe + easy

Suggestion mode throughout both weeks. Humans execute moves and sends. No live connections in this demonstration.

> **Metaphor (restaurant):** POS + kitchen printer + inventory system must all talk, or orders get lost



Guardrails + Edge Cases (Module 3: Deployable, Not Just Demos)

If you can't explain how it fails, you can't responsibly ship it.

Core guardrail

Humans move tickets and send responses. Agent billing and account changes remain prohibited during this pilot.

Security

Least privilege, securely stored keys, audit logs for actions

Quality control

Uncertain or sensitive cases require human review. Confidence does not override permissions or missing evidence.

Edge cases → The incident we will test

Duplicate tickets, conflicting policies, and a timed-out billing lookup. The assistant must not guess which policy applies or promise a credit.

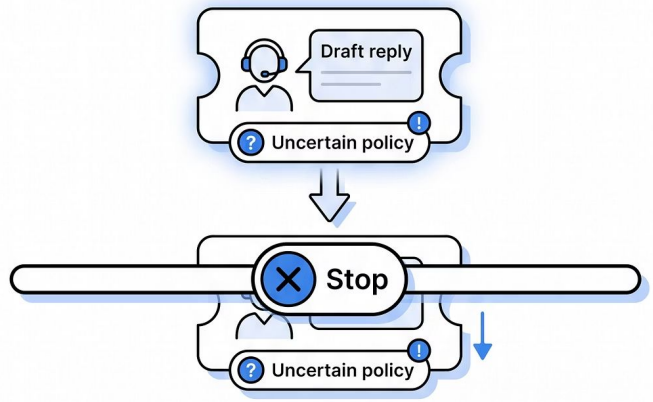
Monitoring

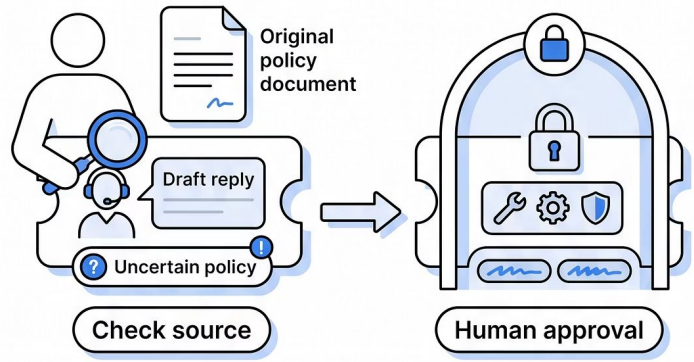
Routing accuracy, override rate, escalation rate, complaint rate, time-to-first-response

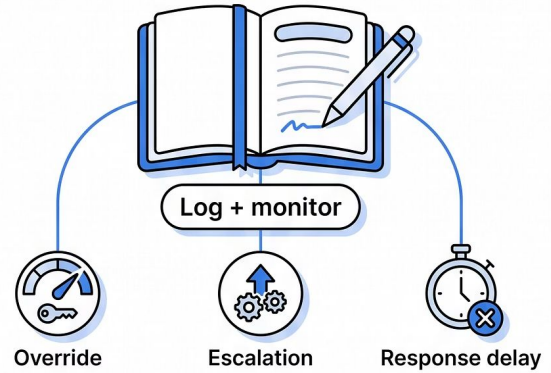
---

Non-technical: guardrails are the rules that keep AI from doing risky things

> **Metaphor (restaurant):** food-safety rules + allergy checks + manager approval before anything "high risk" goes out







Success Metrics + Rollout (Business Value You Can Measure)

A proposal is only credible if it includes 'how we know it worked' and 'how we ship safely.'

Pilot scope (2 weeks)

One ticket category + one team + suggestion mode

KPIs

Exactly Three Fictional Pilot Targets

Average first response 9.4 hours → 6 hours or less

Tickets reassigned at least once 24% → 15% or less

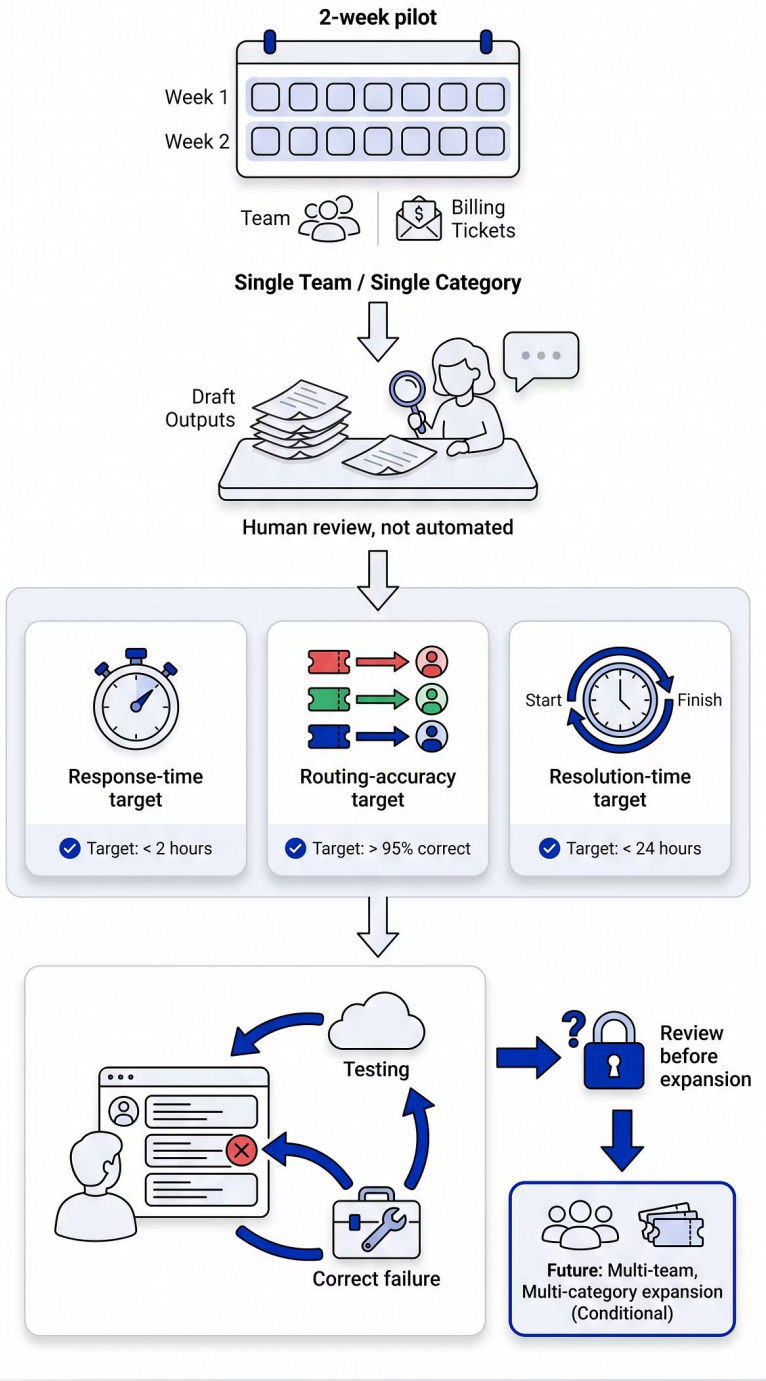
Repeat contact within 72 hours 18% → 13% or less

Iteration loop

Review logs → improve suggestions → retest. No scope or access expansion during the two-week pilot.

> **Metaphor (restaurant):** soft-open with one menu section, track mistakes + wait time, then expand the menu

Non-technical: we test small, measure, then scale what works



Pilot Readiness: Owners, Targets, Kill Switch

A two-week pilot is credible only when responsibility and rollback are explicit.

Owner

Accountable pilot owner: Support Operations Manager.

Human Approvers

People execute ticket moves and external sends. Agent billing and account changes remain outside scope.

Access

Use least privilege. Give the agent access only to approved systems, fields, and actions.

Targets

Choose exactly three KPIs. Write each one as: baseline → target → review cadence.

Kill Switch

Name who can stop the agent, how work returns to people, and where logs remain available for review.

Go/No-Go Rule

A recommendation is not launch permission. Required controls must be validated and launch explicitly authorized.

Metaphor (restaurant): before a soft opening, assign the manager, set service targets, and know how to close the section safely.

